Engenharia de Requisitos no Desenvolvimento Ágil e Generativo

MEI AR 25/26

CPereira_
@isec

1

Modelação ágil

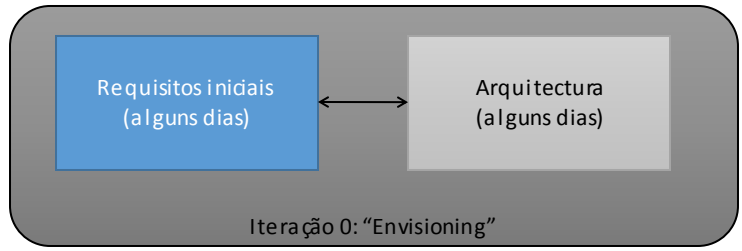
- Passos necessários:
  - Modelação inicial de requisitos,
  - Modelação da iteração,
  - "Model storming"

  - A ideia fundamental é modelar apenas "o suficiente" no início do projecto, de forma a entender os requisitos do sistema a um nível global
  - Os detalhes serão levantados à medida que forem necessários num base Just-In-Time (JIT)

2

Modelação ágil

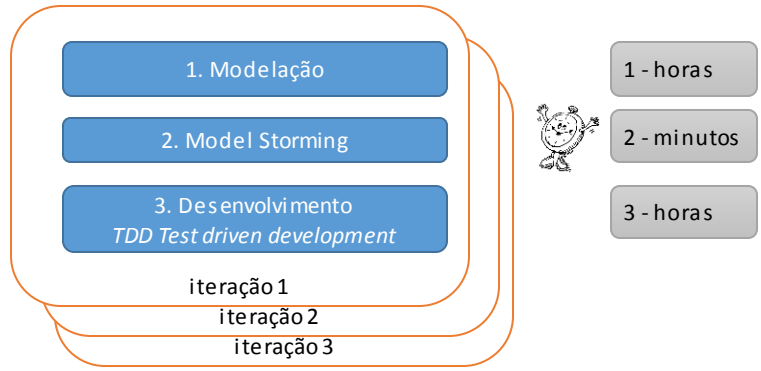
- Processo
  - Identificar visão e âmbito da aplicação
  - Identificar pilha inicial de requisitos
  - Identificar visão da arquitectura



3

Modelação ágil

- Iterações seguintes
  - Modelação: parte importante do processo iterativo. É necessário modelar "o suficiente" para atingir uma boa estimativa do produto final.
  - Planeamento: Planear o trabalho a realizar "dentro" da iteração



4

Requisitos Iniciais

- O objectivo é entender o sistema, não documentar detalhadamente o que o sistema deve fazer.
- Para o modelo de requisitos inicial, são habituais os seguintes artefactos:
  - **Modelo de utilização**
    - Descreve de que forma os utilizadores vão operar o sistema;
    - Pode consistir num conjunto de casos de uso essenciais (num desenvolvimento RUP), num conjunto de features (num projecto Feature Driven Development (FDD)), ou um conjunto de "user stories" (no caso de um projecto Extreme Programming (XP)).

5

Requisitos Iniciais

- **…**
- **Modelo inicial do domínio**
  - Identifica as entidades de negócio fundamentais e relacionamentos entre elas.
  - Pode consistir num conjunto de diagramas de classe UML
  - Contém apenas a informação essencial (diagramas conceptuais): As principais entidades, os atributos principais e as relações.
- **Modelo de interface com o utilizador**
  - Para projectos com uma grande componente de interactividade com o utilizador, incluir "*screen sketches*" ou um protótipo da interface com o utilizador.

6

Iterações

- A modelação da iteração envolve um planeamento das actividades a realizar:
- Deve-se agora explorar com maior detalhe os requisitos, modelando o suficiente para que se possa planear o trabalho necessário para a implementação de um dado requisito.

Model Storming

- Os requisitos de nível superior são analisados num perspectiva "just in time":
  - Como analista, entendi realmente o que é pedido? Se não, perguntar aos stakeholders - "pair programming approach".
  - O requisito é suficientemente pequeno para implementar num sprint? Se não, reorganizar o requisito num conjunto de sub-requsitos, que possam ser tratados individualmente "one at a time".
- Estas sessões de "model storming" não são planeadas, e habitualmente demoram entre 5 a 30 minutos.

Responsabilidades

- Os stakeholders (externos) do projeto, em particular cliente, são a única fonte oficial de requisitos.
- Os membros da equipa de desenvolvimento podem sugerir requisitos, contudo o cliente deve sempre aprová-los.
  - É da inteira responsabilidade dos stakeholders o fornecimento, clarificação, especificação e atribuição de prioridades de requisitos.
  - É da responsabilidade da equipa de desenvolvimento entender e implementar os requisitos.

9

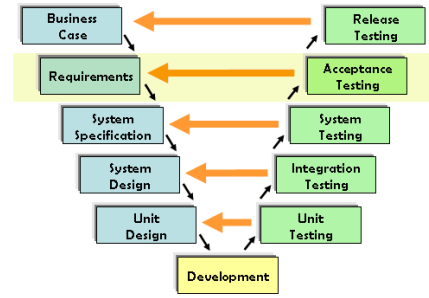
Classes de Requisitos

- **Comportamentais**
  - Descrevem como o utilizador irá interagir com o sistema (aspectos de interface), como alguém irá usar o sistema (aspectos de usabilidade) ou como o sistema realiza uma função de negócio (e.g. como implementa as regras de negócio).
  - São os habitualmente designados requisitos funcionais (nos modelos não ágeis)
- **Não comportamentais**
  - Descrevem características técnicas do sistema: características de desempenho, segurança, fiabilidade, etc.
  - São os habitualmente designados requisitos *não funcionais*.

10

Artefactos

- Teste de aceitação
  - Descreve uma característica (*feature*) observável do sistema, a qual possui interesse para um ou mais stakeholders (teste de cliente)
    - O cliente aprova a implementação apresentada
  - Tipos de requisitos: Funcionais



11

Artefactos

- Regras de negócio
  - Representa um princípio de operação ou política que o sistema deverá satisfazer.
  - Tipos de requisitos: Funcionais
    - Poderá ser detalhada usando o seguinte "template":

- ***Name.***
- ***Description.***
- ***Example (optional).***
- ***Source (optional).***
- ***Related rules (optional).***
- ***Revision history (optional).***

12

Artefactos

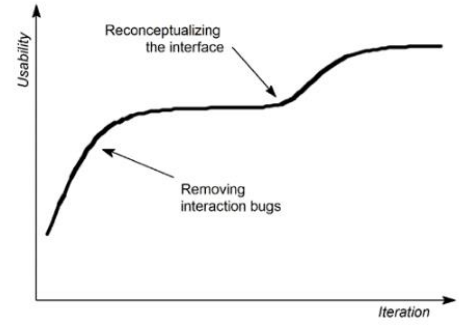
- Alterações "Change case"
  - Descrevem potenciais novos requisitos ou alterações aos requisitos existentes
  - Tipos de requisitos: Ambos (Funcional e Não Funcional)
    - Devem documentar-se. Exemplo:

> - ***Change case:*** *Registration will occur completely via the Internet.*
> - ***Likelihood:*** *Medium likelihood within two to three years, very likely within ten years.*
> - ***Impact:*** *Unknown. Although registration will be available online starting in September, we currently expect less than one quarter of registrations to be made via the Internet this year. Response time will be an issue during the peak use periods, which are the two weeks prior to the beginning of classes each term, as well as the first week of classes.*

13

Artefactos

- Protótipo de Interface com o utilizador
  - Representa apenas os aspectos essenciais do UI
    - Inicialmente trata-se de um modelo ainda pouco fiável – representa uma ideia genérica, não os detalhes.
      - https://www.nngroup.com/articles/iterative-design/



14

Artefactos

- Caso de uso “essencial”
  - Um caso de uso representa uma sequência de acções que fornecem um valor, mensurável, a um actor.
    - Um caso de uso essencial é um caso genérico que representa as funcionalidades de forma independente dos aspectos tecnológicos e de implementação.
  - Tipos de requisitos: Funcionais
  - Também se usam artefactos descrevendo cenários ou um diagrama de casos de uso.

15

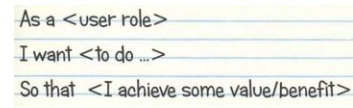
Artefactos

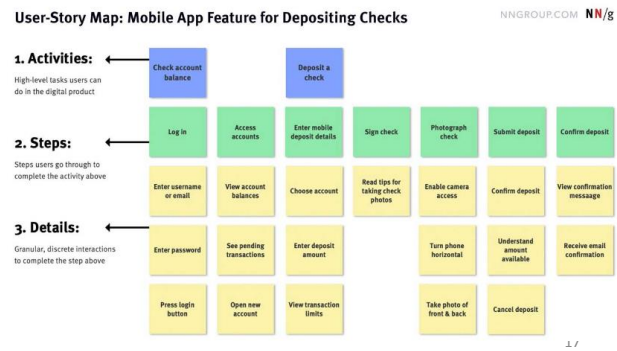
- Feature
  - Neste contexto uma *feature* representa um resultado útil, na perspectiva do cliente
    - (possui uma dimensão pequena, realizável num incremento de duas semanas).
  - Representa usualmente um requisito funcional.

16

Artefactos

- “user story”
  - Funciona como um memorando para conversação com os stakeholders. Capturam os requisitos de alto nível, incluindo requisitos funcionais, não funcionais ou mesmo regras de negócio
  - Criação de Mapas
    - https://www.nngroup.com/articles/user-story-mapping/





17

Práticas

- Participação activa de todos os stakeholders
- Adopção de modelos inclusivos
  - Usar ferramentas muito simples (post-it, quadro branco, cartões, …), de forma a envolver os stakeholders na modelação dos requisitos.
- Obter primeiro a “big picture” – análise horizontal
- Detalhar apenas quando necessário - Just In Time (JIT)
- Dar preferência e especificações executáveis, no lugar de documentação estática
  - O objectivo é implementar requisitos, não documentá-los

18

Práticas

- "Smaller is better"
  - Neste contexto, as user stories são preferíveis aos casos de uso!
  - Usar UML e rastreamento apenas se necessário.
- Explicar as técnicas aos stakeholders
  - Adoptar a terminologia dos stakeholders
  - Integrar os stakeholders na equipa de desenvolvimento
- Estabelecer prioridades no tratamento de requisitos
  - Definida pelo cliente – "business value"
  - Pilha de prioridades

19

Práticas

- **Levantamento dos requisitos**
  - Participação activa dos stakeholders
    - Estende o conceito "on-site customer" para o envolvimento dos stakeholders na modelação de requisitos
  - Entrevistas (directamente, via web, mail, telefone, …)
  - Sessões de brainstormimg
  - Análise de código legado
  - Observação
  - Estudo e leitura

20

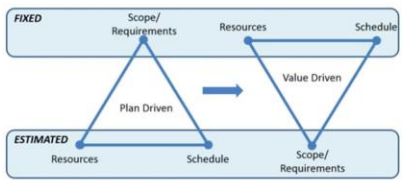
Práticas

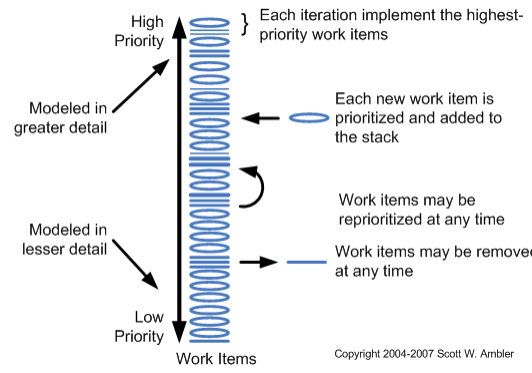
- …
  - Compreenda os objetivos do negócio.
  - Mapeie personas e jornadas.
  - Transforme necessidades em "user stories".
  - Adicione critérios de aceitação claros e passíveis de teste.
  - Refine o backlog de forma contínua.
  - Use protótipos e validação rápida.
  - Priorize pelo valor.

21

Práticas

- Gestão
  - Pilha de prioridades





22

Desafios

- Análise de perceção da Importância das práticas ágeis de engenharia de requisitos [3]

	Practice	Score	Rank Sensitivity* mean	Rank Sensitivity* SD	Tier Stability**
Tier 1	P23: Make requirements testable	25	0.0	0.0	100%
	P03: Establish project shared vision	24	0.0	0.0	100%
	P01: Available/on-site customer	23	0.0	0.0	100%
	P07: Organize review meetings	22	0.0	0.0	97%
	P10: Provide easy access to requirements	20	-0.9	2.1	100%
	P25: Let customer prioritize requirements	19	0.0	0.0	56%
Tier 2	P21: Requirements valuable to purchasers or users	16	0.0	0.0	98%
	P30: Keep release length short	16	0.0	0.5	77%
	P08: Organize retrospective meetings	11	-0.3	0.5	98%
	P22: Make requirements estimable	11	0.0	0.1	100%
	P06: Organize everyday team meetings	9	0.3	1.1	98%
	P31: Define fixed release length	4	0.0	0.0	80%
	P02: Involve different stakeholders	1	-0.7	0.9	79%
Tier 3	P17: Cover requirements with acceptance tests	1	0.0	0.0	96%
	P20: Make complex requirements divisible	1	-0.2	0.9	96%
	P26: Notation easy to understand by all stakeholders	0	-0.1	2.1	91%
	P05: Define project / product constraints	-3	0.0	0.0	92%
	P16: Perform regression acceptance testing	-3	1.9	3.8	95%
	P19: Write short, negotiable requirements	-3	0.0	0.0	99%
	P09: Provide informative workspace	-4	0.0	0.0	96%
	P13: Let customer define acceptance tests	-4	0.0	0.0	99%
	P29: Freeze release scope after it is agreed	-5	0.0	0.0	94%
Tier 4	P28: Negotiate release scope with customer	-9	0.1	0.5	71%
	P27: Assess implementation risks for reqs.	-10	0.0	0.0	90%
	P04: Create prototypes	-19	-0.2	1.6	86%
	P24: Follow the user role modeling approach	-20	0.0	0.0	88%
Tier 5	P18: Make requirements independent	-21	0.3	1.5	64%
	P14: Prepare and maintain automatic acceptance tests	-23	0.2	1.1	94%
	P15: Prepare acceptance tests before coding	-23	0.0	0.0	61%
Tier 6	P11: Maintain "bad smells" and best practices rel. to reqs.	-26	0.0	0.0	71%
Tier 7	P12: Perform the Elevator Test	-30	-3.3	4.3	74%

23

Desafios

- Desafios, causas e potenciais soluções [4]

RE challenges	Resources	Suggestions
Direct communication with stakeholders	[16, 19, 74, 76]	Direct communication, surveys, and interviews
Lack of documentation	[6, 8, 71, 73]	Do some documentation, more consideration, and focus on documentation should be taken
Missing, ambiguous, and conflicting requirements	[16, 75]	Use more formal ways for specification of requirements to incorporate the missing, ambiguous, and conflicting requirements [87]
Problem of prototyping	[68]	Avoid much implementation in initial prototyping, use paper prototyping instead of wizard of Oz prototyping
Less preliminary planning, focus, and no initial team involvement	[16, 81]	Engage and involve all team members of the project from the very beginning such as development team
Less experienced and skilled team	[88–90]	Train practitioner [89], use supportive tools
Inability of customers in telling user stories	[76]	Train customer and involve them in decision making process
Tacit knowledge	[73]	Direct communication, observation, surveys, interviews, and the three factors: domain expert, communication process, and audience will help [40]
Changing requirements	[6, 16, 40, 71, 75]	Use agile requirement change management readiness model (ARCMRM) [91], adopt 21 progress and success factors of ARCM [92], automated techniques, and tools to manage the requirement issues, i.e., JIRA and RE-KOMBINE [40, 71, 87]
Requirement prioritization	[74, 77]	Four stages based frame has been proposed [93]
Negligence of nonfunctional requirements	[40]	Simulation tool NORMATIC for NFRs modeling [67], planning and visualization procedure, [94], NFR elicitation process [95]

Desafios

- Diferenças de ER para processos tradicionais vs ágeis [5]
  - Os desafios da engenharia de requisitos tradicional que são resolvidos pela engenharia de requisitos ágil:

CHALLENGES	DESCRIPTION	HOW AGILE RE DEALS WITH THIS CHALLENGE
Communication Gaps	- Interruption in the provision of the required information to the concerned people ➔ unclear vision of goals and unclear roles	- Frequent face-to-face communication - Collocated teams - Onsite customer - Alternate customer representations - Cross-functional Agile teams - Integrated RE process
Over-Scoping	- Determining the scope of the project that is too large for available resources ➔ It occurs because of communication gaps, scope changes after finalizing a project, and quality issues	- One continuous scope flow - Gradual detailing
Requirements Validation	- It is one of the biggest challenges of traditional requirements engineering	- The continuous prioritization - Requirement changes are suggested and investigated at the end of each iteration - Prototyping
Requirements Documentation	- The reliability of documentation and the accuracy of content ➔ Consuming excessive amounts of time	- Face-to-face communication and tacit knowledge - Documentation is used to include user stories, product backlogs, etc
Customer Involvement	- Rare customer involvement "customers receive the product after development and testing" ➔ Increase the cost of rework and reduce agreement between customers and the development team	- Customers remain aware of the progress during the development life cycle ➔ They provide timely feedback for the development team - Requirements prioritization by the customer

25

Desafios

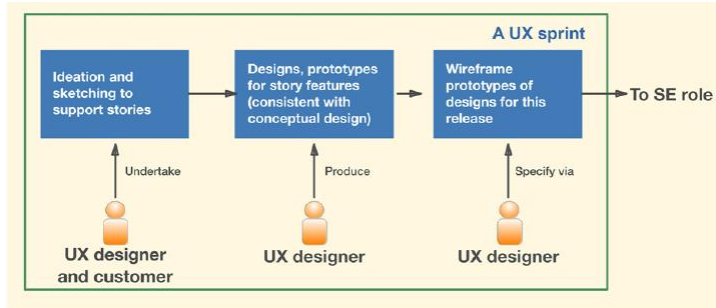
- …
  - Desafios da RE ágil e soluções propostas [5]

CHALLENGE	DESCRIPTION	IMPACT	PROPOSED SOLUTIONS
Minimal Documentation	- Most teams document only user stories and product backlogs	- The absence of customer representatives will arise problems - Difficulties for traceability	- User stories are defined with more detailed artifacts (delivery stories)
Customer Representatives	- The customer availability - The customer competency - The desire of the customer to spend resources on the ongoing verification of requirements and the solution system	- A disagreement between customer groups - Increase in rework	- Alternative or proxy customers to act as real customers (product owners in Scrum) while other organizations implement the "onsite developer" practice - Additional requirements documentation will be beneficial to overcome the overhead created by face-to-face communication - Mind-mapping for requirements elicitation
Imprecise Estimates	- Due to requirements changes and dynamic planning and design, prior accurate estimations for the time and the cost are impossible	- Project delays	- Frequent communication
Inappropriate Architecture	- The initial structure developed by the development team becomes inappropriate in later iterations with new requirements	- Changes occurring during later iterations - Complete rewrite of the system - Increase in cost	- There is no actual efficient solution
Non-Functional Requirements	- Ignoring non-functional requirements with relying on developers implementation on tacit knowledge	- Lower quality ➔ Massive lapse and rework; thus, higher cost and longer time	- Accurate modeling approaches - The NORMATIC tool
Continuous Change of Requirements	- Agile methods allow requirements to change during development	- Increase in cost - Failure of project - Removing misleading and incorrect requirements based on changes requires additional cost and effort	**Contracts:** - Changes must be within agreed restrictions and do not conflict with the contracts concluded - Fixed payment per release protects investments and prevents a random change of requirements **Changes:** - Increase communication and customer engagement - RE-KOMBINE - JIRA
Insufficiency of the User Story Format	- It lacks the power of expression	- The properties such as consistency and verifiability difficult to be validated - Difficulties for traceability	- Delivery stories - Hierarchical requirements model - Aspect-oriented requirements approach - Feature-driven development based requirements analysis process - For traceability issue, traditional requirements analysis, specification process, and requirements traceability management are suggested
Difficulties in the Prioritization of Requirements	- Prioritization of requirements is based on the immediate business value	- Reaching consensus on prioritization of requirements can be difficult due to that customers may have conflicting needs	- Goal-based prioritization technique - Relaying on decision factors like Cost-Benefit to the Organization, Complexity, and Delivery Date/Schedule - Focusing on environment, process, and product - Depending on business value and process flow

26

UX ágil

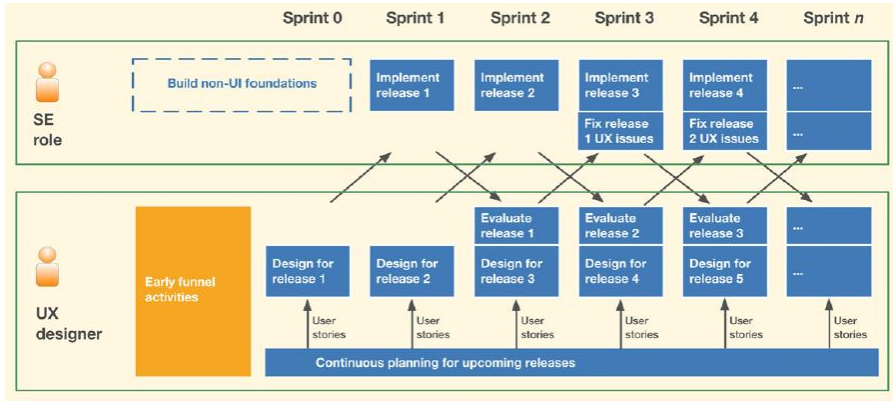
- Processo
  - Enquanto a equipa de desenvolvimento trabalho no sprint, a equipa de UX e o cliente executa a sua própria "versão de um sprint" [7]:



27

UX ágil

- …



28

UX ágil

- Processo de trabalho - um cenário:
  - UX:
    - Visualizar "user stories" – JIRA,…
    - Modelação e "user research" – BR, outras apps, definir terminologia, modelo de dados
    - Idealização e construção do esboço inicial
    - Construir wireframes – Sketch,…
    - Construir Protótipo funcional (navegação) – Invision, Craft
    - Construir Traceability – "user story" para protótipo (JIRA)

29

UX ágil

- …
  - Com toda a equipa (UX):
    - revisão do projeto e iterar até obtenção de um acordo (da UX)
  - Com a equipa de DEV:
    - revisão do projeto inicial (pré-implementação)
    - Atualizar o protótipo (wireframe) com as alterações da equipa DEV
    - Caso necessário, aumentar a fidelidade do protótipo.
  - Com outras partes interessadas:
    - Revisão do projeto, iterar conforme necessário
  - DEV:
    - Revisão e passagem do projeto à equipa UX
  - UX:
    - Criar "issue" e atribuir à equipa DEV, atualizar "link" para InVision (JIRA)
  - DEV:
    - Criar e atribir JIRA issue a UX, criar link de dependência para versão original)

30

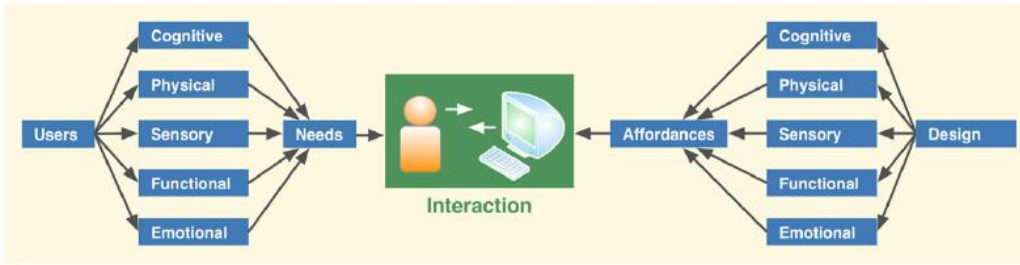
UX ágil

- ….
  - **QA - Controlo de qualidade**: verificação de fidelidade da implementação - O que é construído deve refletir exatamente o protótipo.
    - Se não houver discrepância,
      - terminar (UX)
    - Caso contrário:
      - UX – criar JIRA issue "Atualização de software com revisão de UX opinião"
    - Repetir até não haver discrepância

31

UX ágil

- Usabilidade
  - As necessidades de interação com o utilizador estão relacionadas diretamente às ofertas e disponibilidade do projeto:



32

UX ágil

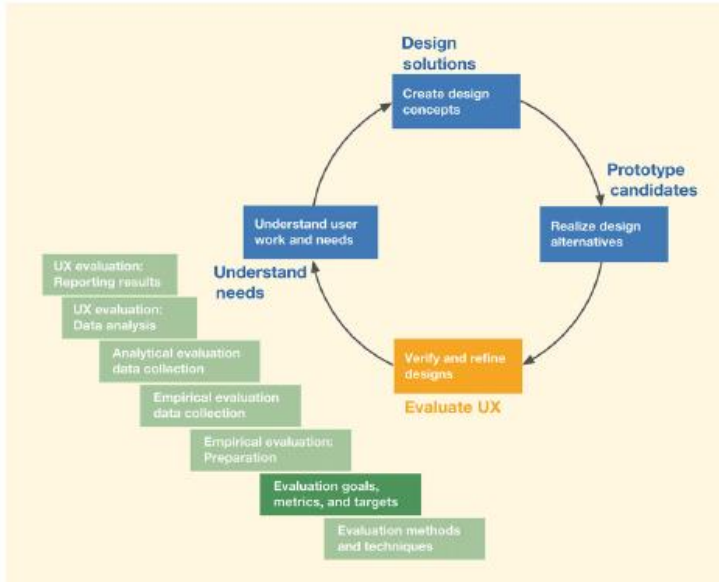
- …

Affordance Type	Description	Example
Cognitive affordance	Design feature that helps users with their cognitive actions: Thinking, deciding, learning, remembering, and knowing about things	A button label that helps users know what will happen if they click on it
Physical affordance	Design feature that helps users with their physical actions: Clicking, touching, pointing, gesturing, and moving things	A button that is large enough so that users can click on it accurately
Sensory affordance	Design feature that helps users with their sensory actions: Seeing, hearing, and feeling (and tasting and smelling) things	A label font size large enough to be discerned
Functional affordance	Design feature that helps users employ a product or system to accomplish work (i.e., usefulness of a system function)	The internal system ability to sort a series of numbers (invoked by users clicking on the Sort button)
Emotional affordance	Design feature that adds emotional impact to the user experience and helps users appreciate and enjoy the interaction	Beautiful aesthetics on a webpage, something that makes interaction fun

33

UX ágil

- Avaliação



34

UX ágil

- …

Informal Summative UX Evaluation
Engineering
Deliberately nonrandom participant selection to get most formative information
Deliberately uses relatively small number of participants
Deliberately simple, low-power statistical techniques (e.g., simple mean and, sometimes, standard deviation)
Results cannot be used to make claims, but are used to make engineering judgments
Relatively inexpensive and rapid to perform
Methods and procedures open to innovation and adaptation
Can yield insight about broader range of questions regarding levels of UX achieved and the need for further improvement
Intended to be used within a UX design process in support of formative methods

Work Role: User Class	UX Goal	UX Measure	Measuring Instrument	UX Metric	Baseline Level	Target Level	Observed Results	Meet Target?
Ticket buyer: Casual new user, for occasional personal use	Walk-up ease of use	Initial user performance	BT1: Buy special event ticket	Average time on task	3 min as measured at the kiosk	2.5 min	3.5 min	No
Ticket buyer: Casual new user, for occasional personal use	Walk-up ease of use for new user	Initial user performance	BT2: Buy movie ticket	Average number of errors	<1	<1	2	No
Ticket buyer: Casual new user, for occasional personal use	Initial customer satisfaction	First impression	Questions Q1–Q10 in questionnaire XYZ	Average rating across users and across questions	7.5/10	8/10	7.5	No

35

UX ágil

- …
  - Impacto Emocional
    - A utilização foi "divertida"?
    - O aspeto visual é atraente?
      - por exemplo, cores, formas, layout
    - O design é criativo?



36

UX ágil

- …
  - Tabela de importância de custo:

Importância para corrigir:
- "Must"
- Grande impacto 5,
- Moderado 3,
- Baixo 1

Custo para corrigir – no estado atual de desenvolvimento
Normalmente são calculados com base nos recursos necessários (pessoa/hora)

Priority ratio= =importance/cost*1000

Problem	Imp.	Solutions	Cost	Prio. Ratio	Prio. Rank	Cuml. Cost	Resolution
Didn't recognize the "counter" as being for the number of tickets. As a result, user failed to even think about how many tickets he needed	*M*	Move quantity information and label it	2	*M*	1	2	Fix in this version
User unaware of the need to click on the "Submit" button to proceed to payment	4	Change the label wording to "Proceed to Payment"	1	4000	2	3	Fix in this version
User confused about "Theatre" on the "Choose a domain" screen. Thought it meant choosing a physical theater (as a venue) rather than the category of theatre arts	3	Improve the wording to "Theatre Arts"	1	3000	3	4	Fix in this version
Unsure of current date and what date he was purchasing tickets for	5	Add current date field and label all dates precisely	2	2500	4	6	Fix in this version
Didn't like having a "Back" button on second screen because first screen was only a "Welcome"	2	Remove it	1	2000	5	7	Fix in this version
Users were concerned about their work being left for others to see	5	Add a timeout feature that clears the screens	3	1667	6	10	Fix in this version

Priority ratio= M, se Imp=M

Resolution: decidir como cada problema será tratado!

Custo acumulado: custo de corrigir cada problema mais o custo de todos os problemas acima dele na tabela.

37

UX ágil

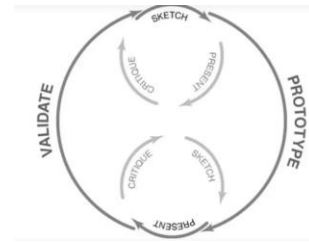
- …
  - Ações consequentes – exemplo [7]:

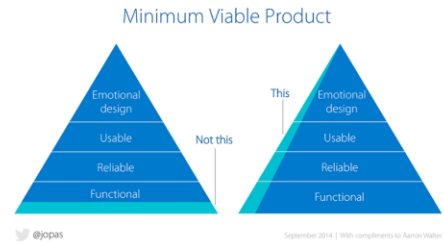
Seat category distinguishability usability problem	9. User expected graphic of seat layout but missed seeing the button for that at first; kept missing "View Seats"	Group "View Seats" button in layout with other seat purchase task controls
	13. For "Selected seats," no way to distinguish balcony from floor seats because they use the same numbering scheme	Distinguish balcony seats and floor seats with different numbering schemes
	20. In "View Seats" view, participant couldn't determine which seats were already sold, because color choices were confusing	Use different icons or colors to distinguish which seats are already sold
	25. Missed fact that blue seat sections are clickable to zoom in on detailed view of available seats. User left thinking that blue seat section wasn't enough information about *which* seats are available	Show clearly that blue seat sections are clickable. When one is clicked, display the detailed seat information, like location, price, etc. Add legend to explain color usage

38

Lean UX

- Processo
  - Lean Code
    - Código simples e direto
    - DRY "dont repeat yourself"
  - Lean UX
    - "Design fast, test early, learn continuosly"





"**UX Must Work at Least One Step Ahead of the Sprint"**

https://www.nngroup.com/articles/doing-ux-agile-world/

39

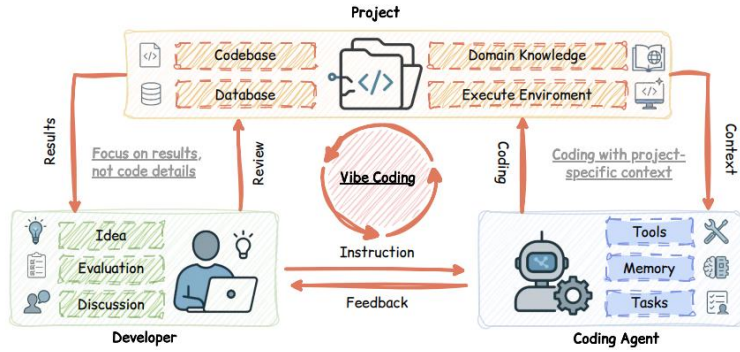
Lean UX

- Processo Lean - "Maximizing user value through continuous experimentation"
  - Ciclos de 4 etapas [6-8]:
    - Suposições
      - Identificação do Problema
        - Tabelas (features/suposição/solução)
      - Hipóteses
      - Personas
      - Lista de Tarefas
        - Tabela "Feature/Task/Steps"
      - Design Colaborativo
        - Criar um sketch "low fidelity prototype"
    - Criar um MVP *minimum viable product*
      - "high fidelity prototype"
    - Executar – Experimentar
      - Avaliar o MVP (com stakeholders); preencher tabela "Experiência/Tarefa/resultado"
    - Feedback e Pesquisa
      - Validar o MVP através de testes de usabilidade; tabela "Hipótese/user feedback"



40

Vibe coding



- https://arxiv.org/pdf/2510.12399

41

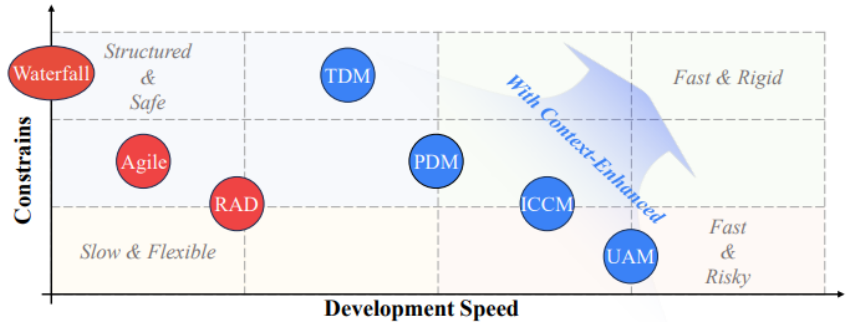
Vibe coding

- …
  - Nível de colaboração entre Humano e IA Generativa "Human in the Loop"
    - **UAM** Unconstrained Automation Model
      - Confiança total nos resultados da IA, com uma análise mínima do código e validação da sua correção através de testes de funcionalidade, em vez da sua compreensão.
      - Semelhança com as metodologias de Desenvolvimento Rápido de Aplicações (RAD) na engenharia de software tradicional, que também enfatizam a criação de protótipos de forma rápida e ciclos de desenvolvimento mais curtos.
    - **ICCM** Iterative Conversational Collaboration Model
      - IAG como parceira de programação, em vez de agente totalmente autónoma
    - **PDM** – Plan Driven Modeling
      - Humano define e envia os requisitos (SRS, PRD,…)
    - **TDM** – Test Driven Modeling
      - Primeiro escrevem-se os testes de aceitação
      - IAG gera código para os testes "passarem"

42

Vibe coding

- …



- Modelos de desenvolvimento de codificação Vibe vs Modelos tradicionais. https://arxiv.org/pdf/2510.12399

43

Referências

- [1] Agile Modeling: Effective Practices for Extreme Programming and the Unified Process, Scott W. Ambler , Ron Jeffries
- [2] http://www.agilemodeling.com
- [3] Ochodek, M., & Kopczyńska, S. (2018). Perceived importance of agile requirements engineering practices–a survey. *Journal of Systems and Software*, *143*, 29-43.
- [4] Rasheed, A., Zafar, B., Shehryar, T., Aslam, N. A., Sajid, M., Ali, N., ... & Khalid, S. (2021). Requirement Engineering Challenges in Agile Software Development. *Mathematical Problems in Engineering*, *2021*.
- [5] Alhazmi, A., & Huang, S. (2020, March). Survey on Differences of Requirements Engineering for Traditional and Agile Development Processes. In *2020 SoutheastCon* (pp. 1-9). IEEE.
- [6] Hidayah, N. A., Rafiiuddin, A., Durachman, Y., & Rustamaji, E. (2021, September). User Experience Design Analysis Using Lean UX Method. In *2021 9th International Conference on Cyber and IT Service Management (CITSM)* (pp. 1-6). IEEE.
- [7] Hartson, R., & Pyla, P. S. (2018). *The UX book: Agile UX design for a quality user experience*. Morgan Kaufmann.
- [8] https://www.nngroup.com/articles/lean-ux-agile-study-guide/
- [9] https://arxiv.org/pdf/2510.12399

44